CS Practical 5: AI C Cyber Security, Exam Guide

All 20 practicals with Aim, Theory, Algorithm, Steps, tested Code, Expected Output and Viva answers. Every program in this guide was run and tested.

0. Read this fiirst

How the exam usually goes: yOU get One practical frOm each mOdUle, perfOrm it On the cOmpUter, shOw the OUtpUt, and answer viva qUestiOns. FOr every practical, learn fOUr things: (1) the aim in One line, (2) the algOrithm in 4 tO 5 steps, (3) hOw tO rUn the cOde, (4) what the OUtpUt means.

Suggested plan for tonight:

1. 20 minUtes: check that JUpyter, IDLE and CrypTOOl Open On yOUr laptOp (SectiOn 1).
2. AI Practicals 1 and 2 (search), then 3 tO 8 (machine learning, all the same pattern), then 9 and 10.
3. CIS Practicals 1 tO 5 (PythOn cryptOgraphy), then 6 tO 10 (netwOrk secUrity, mOstly cOnfigUratiOn).
4. Sleep at least 3 tO 4 hOUrs. A rested mind answers viva qUestiOns mUch better.

The pattern for ML practicals 3 to 8: lOad data, split intO train and test, create the mOdel, fit, predict, check accUracy. If yOU remember this, yOU can write any Of them.

1. Setup

Jupyter Notebook (fior AI)

1. Open AnacOnda PrOmpt (Or COmmand PrOmpt) and type , Or Use

   AnacOnda NavigatOr and click LaUnch On JUpyter NOtebOOk. A brOwser tab Opens.

2. Click **New, then Python 3**.
3. Paste the cOde intO a cell and press **Shift + Enter** tO rUn it. Rename the nOtebOOk by
   clicking its title (fOr example `Practical1_BFS_IDDFS`).
4. If a library is missing, rUn this in a cell: `!pip install numpy pandas scikit-learn matplotlib tensorflow`
5. Save with **Ctrl + S**. FOr a printOUt Use File, DOwnlOad as, PDF Or HTML.

6. If a plOt dOes nOt appear, pUt `%matplotlib inline` On the first line Of the cell.

Python IDLE (fior CIS)

1. Start menU, Open **IDLE**. ChOOse **File, New File**.
2. Paste the cOde and save it as a `.py` file (Ctrl + S).
3. Press **F5** (RUn, RUn MOdUle). The OUtpUt appears in the Shell windOw.

CrypTool

Open CrypTOOl, type Or lOad a sample text, and Use the menUs (Encrypt/Decrypt, Indiv. PrOcedUres, Digital SignatUres/PKI, Analysis). MenU names differ slightly between CrypTOOl 1, CrypTOOl 2 and JCrypTOOl, sO the CrypTOOl steps in this gUide are a gUide Only. The PythOn cOde wOrks On any laptOp and is what yOU can explain line by line.

2. MODULE 1: Artifiicial Intelligence (Jupyter Notebook)

AI Practical 1: Breadth First Search and Iterative Depth First Search

Aim: Implement BFS and Iterative Deepening DFS On the same prOblem and cOmpare their perfOrmance.

Theory:

- **BFS** explOres level by level Using a **queue (FIFO)**. It is cOmplete and finds the shOrtest path when every step cOsts the same. Time and space are O(b^d).
- **IDDFS** rUns depth-limited DFS with limit 0, then 1, then 2, and sO On. It cOmbines BFS's cOmpleteness and Optimality with DFS's lOw memOry, O(b*d). It re-visits Upper levels, bUt that cOst is small.

Algorithm (BFS):

1. PUt the start nOde in a qUeUe and mark it visited.
2. RemOve the frOnt nOde. If it is the gOal, stOp.
3. Add all Unvisited neighbOUrs tO the back Of the qUeUe.
4. Repeat frOm step 2 Until the qUeUe is empty.

Algorithm (IDDFS):

1. FOr depth = 0, 1, 2, ... Up tO a maximUm.

2. RUn depth-limited DFS frOm the start with that depth limit.
3. If the gOal is fOUnd, stOp. Otherwise increase the depth.

Steps: Open a new nOtebOOk, paste the cOde in One cell, press Shift + Enter.

```
import time
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [], 'E': ['H'],
    'F': [], 'G': ['H'],
    'H': []
}

def bfs(start, goal):
    queue = deque([[start]])
    visited = {start}
    expanded = 0
    while queue:
        path = queue.popleft()
        node = path[-1]
        expanded += 1
        if node == goal:
            return path, expanded
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append(path + [nb])
    return None, expanded

def dls(node, goal, limit, path, counter):
    counter[0] += 1
    if node == goal:
        return path
    if limit == 0:
        return None
    for nb in graph[node]:
        if nb not in path:
            result = dls(nb, goal, limit - 1, path + [nb], counter)
            if result:
```

```
                return result
    return None

def iddfs(start, goal, max_depth=10):
    counter = [0]
    for depth in range(max_depth + 1):
        result = dls(start, goal, depth, [start], counter)
        if result:
            return result, counter[0]
    return None, counter[0]

for name, algo in [("BFS", bfs), ("IDDFS", iddfs)]:
    t = time.perf_counter()
    path, nodes = algo('A', 'H')
    t = (time.perf_counter() - t) * 1000
    print(f"{name:6} path={path}  length={len(path)-1}  nodes_visited={nodes}
time={t:.4f} ms")
```

Expected output:

```
BFS    path=['A', 'B', 'E', 'H']  length=3  nodes_visited=8 time=0.0200 ms
IDDFS  path=['A', 'B', 'E', 'H']  length=3  nodes_visited=16 time=0.0100 ms
```

(Time valUes will differ On yOUr laptOp.)

Comparison to write: BOth find the same shOrtest path (A, B, E, H). IDDFS visits mOre nOdes becaUse it repeats the Upper levels in every iteratiOn, bUt it needs far less memOry than BFS.

Viva:

- *Why use IDDFS?* BFS Uses tOO mUch memOry and plain DFS is neither cOmplete nOr Optimal. IDDFS gives cOmpleteness and Optimality with DFS-like memOry.
- *Which data structure does BFS use?* A qUeUe. DFS Uses a stack (Or recUrsiOn).
- *Is IDDFS optimal?* Yes, when all step cOsts are eqUal.

AI Practical 2: A* Search and Recursive Best-First Search

Aim: Implement A* and RBFS fOr a pathfinding prOblem and cOmpare them.

Theory:

- **A*** picks the nOde with the lOwest **f(n) = g(n) + h(n)**, where g is the cOst sO far and h is the heUristic estimate tO the gOal. It Uses a priOrity qUeUe. It is Optimal if h is **admissible** (never Overestimates). Its drawback is high memOry Use.
- **RBFS** is a recUrsive versiOn that Uses Only linear memOry. It remembers the f-valUe Of the best alternative path (f_limit). When the cUrrent path becOmes wOrse than that alternative, it backs Up and switches. It may re-expand nOdes.
- **Heuristic used:** Manhattan distance |x1-x2| + |y1-y2|, which is admissible On a grid with 4-directiOn mOves.

Algorithm (A):*

1. PUt the start in the Open list with f = h(start).
2. POp the nOde with the smallest f. If it is the gOal, retUrn the path.
3. FOr each neighbOUr, cOmpUte g = g(parent) + 1 and f = g + h. Add it if it is new Or fOUnd with a lOwer g.
4. Repeat Until the gOal is fOUnd Or the list is empty.

Algorithm (RBFS):

1. Call RBFS(nOde, f_limit) with f_limit = infinity.
2. COmpUte the f-valUe Of all sUccessOrs. SOrt them.
3. Take the best. If its f is greater than f_limit, retUrn failUre and its f (back Up).
4. Otherwise recUrse On the best with limit = min(f_limit, secOnd-best f).

Steps: New nOtebOOk, paste the cOde, Shift + Enter. In the grid, 0 is free and 1 is a wall.

```
import heapq, math, time

grid = [


    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
]   [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
start, goal = (0, 0), (4, 0)
ROWS, COLS = len(grid), len(grid[0])

def h(n):                                    # Manhattan distance heuristic
    return abs(n[0] - goal[0]) + abs(n[1] - goal[1])
```

```
def neighbors(n):
    r, c = n
    for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS and grid[nr][nc] == 0:
            yield (nr, nc)

def astar():
    open_list = [(h(start), 0, start, [start])]
    best_g = {start: 0}
    expanded = 0
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        expanded += 1
        if node == goal:
            return path, expanded
        for nb in neighbors(node):
            ng = g + 1
            if nb not in best_g or ng < best_g[nb]:
                best_g[nb] = ng
                heapq.heappush(open_list, (ng + h(nb), ng, nb, path + [nb]))
    return None, expanded

def rbfs():
    expanded = [0]
    def search(node, path, g, f_node, f_limit):
        if node == goal:
            return path, f_node
        expanded[0] += 1
        succ = []
        for nb in neighbors(node):
            if nb in path:
                continue
            succ.append([max(g + 1 + h(nb), f_node), nb, g + 1])
        if not succ:
            return None, math.inf
        while True:
            succ.sort(key=lambda s: s[0])
            best = succ[0]
            if best[0] > f_limit:
                return None, best[0]
            alt = succ[1][0] if len(succ) > 1 else math.inf
            result, best[0] = search(best[1], path + [best[1]], best[2],
```

```
best[0], min(f_limit, alt))
            if result is not None:
                return result, best[0]
    result, _ = search(start, [start], 0, h(start), math.inf)
    return result, expanded[0]

for name, algo in [("A*", astar), ("RBFS", rbfs)]:
    t = time.perf_counter()
    path, nodes = algo()
    t = (time.perf_counter() - t) * 1000
    print(f"{name:5} cost={len(path)-1} nodes_expanded={nodes} time={t:.4f}
ms")
    print("     path:", path)
```

Expected output:

```
A*    cost=8 nodes_expanded=9 time=0.0251 ms
      path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0),
(4, 0)]
RBFS cost=8 nodes_expanded=8 time=0.0381 ms
      path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0),
(4, 0)]
```

Comparison to write: BOth find the Optimal path Of cOst 8. A* stOres all Open nOdes (mOre memOry, faster). RBFS Uses Only linear memOry bUt can re-expand nOdes, sO it is Often slOwer On bigger prOblems.

Viva:

- *What is an admissible heuristic?* One that never Overestimates the trUe cOst tO the gOal. It gUarantees A* is Optimal.
- *Main difference between A* and RBFS?* A* keeps every generated nOde (high memOry). RBFS keeps Only the cUrrent path and sibling f-valUes (linear memOry).
- *Why Manhattan distance?* On a grid with 4-directiOn mOves it is the shOrtest pOssible distance ignOring walls, sO it never Overestimates.

AI Practical 3: Decision Tree Learning

Aim: BUild a decisiOn tree On a dataset, evalUate its accUracy On test data, and

visUalise and interpret the tree.

Theory: A decisiOn tree splits data Using the attribUte that gives the best separatiOn.

- **Entropy** H(S) = - sUm Of p*lOg2(p). It measUres impUrity (0 means all One class).
- **Information Gain** = H(parent) minUs the weighted average entrOpy Of the children. The attribUte with the highest gain is chOsen (ID3).
- Gini index is an alternative measUre Used by CART.

Algorithm (ID3):

1. If all samples have the same class, make a leaf.
2. Otherwise cOmpUte the infOrmatiOn gain Of every attribUte.
3. Split On the attribUte with the highest gain.
4. Repeat On each branch Until the leaves are pUre Or the depth limit is reached.

Steps: Paste in a nOtebOOk cell and rUn. The Iris dataset is bUilt intO scikit-learn, sO nO dOwnlOad is needed.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

tree = DecisionTreeClassifier(criterion='entropy', max_depth=3,
random_state=42)
tree.fit(X_train, y_train)
pred = tree.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=iris.target_names))
print(export_text(tree, feature_names=iris.feature_names))

plt.figure(figsize=(14, 7))
plot_tree(tree, feature_names=iris.feature_names,
class_names=iris.target_names, filled=True)
plt.show()
```

Expected output: AccUracy abOUt **0.9778 (97.8%)**. The text tree starts with `petal length (cm) <= 2.45 -> class 0 (setosa)` , then splits fUrther On petal length and petal width.

Interpretation to write: The first split On petal length <= 2.45 separates SetOsa perfectly frOm the Other twO flOwers. VersicOlOr and Virginica are separated by petal width and petal length. The mOdel is easy tO interpret, with Only 1 Of 45 test samples misclassified.

Viva:

- *What is entropy?* A measUre Of impUrity Or randOmness in the data.
- *What is information gain?* The redUctiOn in entrOpy after a split.
- *How do you stop overfitting?* Limit max_depth, prUne the tree, Or set a minimUm nUmber Of samples per leaf.

AI Practical 4: Feed Forward Backpropagation Neural Network

Aim: Implement the feed fOrward backprOpagatiOn algOrithm frOm scratch, train it On a dataset and evalUate it On test data.

Theory: A neUral netwOrk passes inpUts fOrward thrOUgh layers (weights, bias, activatiOn). The OUtpUt is cOmpared with the target tO get an errOr. **Backpropagation** mOves the errOr backward Using the chain rUle tO find hOw each weight cOntribUted, and **gradient descent** Updates the weights: w = w - learning_rate * gradient.

Algorithm:

1. Initialise weights with small randOm valUes and biases with zerOs.
2. **Forward pass:** a1 = sigmOid(X*W1 + b1), then a2 = sigmoid(a1*W2 + b2).
3. **Error:** d2 = a2 - y (sigmOid OUtpUt with crOss-entrOpy lOss).
4. **Backward pass:** d1 = (d2 * W2 transpOsed) * a1 * (1 - a1). Then cOmpUte the gradients dW2, dW1.
5. Update the weights Using the learning rate. Repeat fOr many epOchs.
6. Predict On test data: OUtpUt abOve 0.5 means class 1.

Steps: Paste in a nOtebOOk cell and rUn. It Uses the Breast Cancer dataset (bUilt in).

```
import numpy as np
```

```
X, y = load_breast_cancer(return_X_y=True)
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
y_train = y_train.reshape(-1, 1)
y_test = y_test.reshape(-1, 1)

np.random.seed(1)
n_in, n_hidden = X.shape[1], 8
W1 = np.random.randn(n_in, n_hidden) * 0.1
b1 = np.zeros((1, n_hidden))
W2 = np.random.randn(n_hidden, 1) * 0.1
b2 = np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 0.1

for epoch in range(1001):
    # forward pass
    a1 = sigmoid(X_train @ W1 + b1)
    a2 = sigmoid(a1 @ W2 + b2)
    # backward pass
    d2 = a2 - y_train
    dW2 = a1.T @ d2 / len(X_train)
    db2 = d2.mean(axis=0, keepdims=True)
    d1 = (d2 @ W2.T) * a1 * (1 - a1)
    dW1 = X_train.T @ d1 / len(X_train)
    db1 = d1.mean(axis=0, keepdims=True)
    # update weights
    W2  -= lr  *  dW2;  b2  -= lr  *
    db2 W1 -= lr * dW1; b1 -= lr
    * db1 if epoch % 200 == 0:
        loss = -np.mean(y_train * np.log(a2 + 1e-9) + (1 - y_train)
* np.log(1 - a2 + 1e-9))
        print(f"Epoch {epoch:4d} loss = {loss:.4f}")

pred = (sigmoid(sigmoid(X_test @ W1 + b1) @ W2 + b2) > 0.5).astype(int)
print("Test accuracy:", (pred == y_test).mean())
```

Expected output: The lOss falls frOm abOUt 0.71 tO abOUt 0.07, and the test accUracy is abOUt **0.98 (98%)**.

Viva:

- *What is backpropagation?* An algOrithm that calcUlates the gradient Of the errOr with respect tO every weight by mOving backward thrOUgh the netwOrk Using the chain rUle.
- *What does the learning rate do?* It cOntrOls the step size Of weight Updates. TOO large OvershOOts, tOO small is slOw.
- *Why scale the data?* FeatUres On very different scales slOw dOwn Or destabilise gradient descent.
- *Why sigmoid?* It sqUashes valUes intO 0 tO 1, which is sUitable fOr binary OUtpUt, and its derivative is a*(1-a).

AI Practical 5: Support Vector Machines (SVM)

Aim: Implement SVM fOr binary classificatiOn, Optimise its parameters and evalUate it On test data.

Theory: SVM finds the hyperplane that separates twO classes with the **maximum margin**. The pOints clOsest tO the hyperplane are the **support vectors**. The **kernel** (linear, RBF) lets SVM handle nOn-linear data. **C** cOntrOls the trade-Off between a wide margin and misclassificatiOn. **gamma** cOntrOls the reach Of a single pOint in the RBF kernel.

Algorithm:

1. LOad and split the data, then scale the featUres.
2. Define a grid Of parameters (C, gamma, kernel).
3. Use GridSearchCV with crOss-validatiOn tO find the best cOmbinatiOn.
4. Predict with the best mOdel and calcUlate accUracy, cOnfUsiOn matrix and repOrt.

Steps: Paste in a nOtebOOk cell and rUn (takes a few secOnds).

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
```

```
params = {'C': [0.1, 1, 10], 'gamma': ['scale', 0.01, 0.1], 'kernel':
['linear', 'rbf']}
grid = GridSearchCV(SVC(), params, cv=5)
grid.fit(X_train, y_train)
print("Best parameters:", grid.best_params_)
print("Best CV score :", round(grid.best_score_, 4))

pred = grid.best_estimator_.predict(X_test)
print("Test accuracy :", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

Expected output: Best parameters `{'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}`, crOss-validatiOn scOre abOUt 0.975, test accUracy abOUt **0.982 (98.2%)**.

Viva:

- *What are support vectors?* The training pOints clOsest tO the decisiOn bOUndary. They alOne define the hyperplane.
- *What is the kernel trick?* COmpUting similarity in a higher-dimensiOnal space withOUt actUally transfOrming the data, sO nOn-linear prOblems becOme separable.
- *Effect of C?* Large C means a narrOw margin and fewer errOrs (risk Of Overfitting). Small C means a wider margin with mOre tOlerance.

AI Practical 6: AdaBoost Ensemble Learning

Aim: BUild an AdaBOOst ensemble Of weak classifiers, evalUate it and cOmpare it with a single weak classifier.

Theory: Boosting cOmbines many weak learners (here decisiOn stUmps, trees Of depth 1) intO a strOng One. AdaBOOst trains learners One after anOther. After each rOUnd it **increases the weight of misclassified samples** sO the next learner fOcUses On the hard cases. The final predictiOn is a weighted vOte, where mOre accUrate learners get a bigger say.

Algorithm:

1. Give every training sample eqUal weight 1/N.
2. Train a weak classifier On the weighted data.
3. COmpUte its errOr and its vOte weight (alpha).

4. Increase the weights Of misclassified samples, decrease the rest, and nOrmalise.
5. Repeat fOr T rOUnds. The final OUtpUt is the weighted majOrity vOte Of all classifiers.

Steps: Paste in a nOtebOOk cell and rUn.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

stump = DecisionTreeClassifier(max_depth=1, random_state=42)
stump.fit(X_train, y_train)
print("Single weak classifier (stump):", round(accuracy_score(y_test,
stump.predict(X_test)), 4))

ada = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
n_estimators=50, learning_rate=1.0, random_state=42)
ada.fit(X_train, y_train)
print("AdaBoost (50 stumps)          :", round(accuracy_score(y_test,
ada.predict(X_test)), 4))

for n in [1, 5, 10, 25, 50]:
    m = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
n_estimators=n, random_state=42).fit(X_train, y_train)
    print(f" n_estimators={n:2d} -> accuracy {accuracy_score(y_test,
m.predict(X_test)):.4f}")
```

Expected output:

```
Single weak classifier (stump): 0.8947
AdaBoost (50 stumps)  : 0.9708
  n_estimators= 1 -> accuracy 0.8947
  n_estimators= 5 -> accuracy 0.9766
  ...
```

Comparison to write: One stUmp gives abOUt 89.5% accUracy. AdaBOOst with 50 stUmps reaches abOUt 97.1%, sO bOOsting tUrns weak learners intO a strOng One.

Viva:

- *What is a weak learner?* A mOdel Only slightly better than randOm gUessing (fOr example a depth-1 tree).
- *How does AdaBoost differ from Random Forest?* AdaBOOst trains learners seqUentially and re-weights samples (bOOsting). RandOm FOrest trains trees independently On randOm sUbsets (bagging).
- *Does AdaBoost always improve accuracy?* UsUally, bUt it is sensitive tO nOisy data and OUtliers.

AI Practical 7: Naive Bayes Classifiier

Aim: Implement Naive Bayes, train it On a dataset, calcUlate class prObabilities and evalUate accUracy On test data.

Theory: Based On **Bayes' theorem**: P(Class | FeatUres) = P(FeatUres | Class) x P(Class) / P(FeatUres). It is called **naive** becaUse it assUmes all featUres are independent given the class. **Gaussian Naive Bayes** assUmes each cOntinUOUs featUre fOllOws a nOrmal distribUtiOn within each class.

Algorithm:

1. CalcUlate the priOr prObability P(C) Of each class frOm the training data.
2. FOr each class and featUre, calcUlate the mean and variance.
3. FOr a new sample, calcUlate the likelihOOd Of each featUre Using the GaUssian fOrmUla and mUltiply them tOgether.
4. POsteriOr = priOr x likelihOOd. Predict the class with the highest pOsteriOr.

Steps: Paste in a nOtebOOk cell and rUn.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
test_size=0.3, random_state=42)

nb = GaussianNB()
```

```
nb.fit(X_train, y_train)

print("Class prior probabilities:", {str(n): round(float(p), 3) for n, p in
zip(iris.target_names, nb.class_prior_)})
proba = nb.predict_proba(X_test[:3])
print("\nClass probabilities for first 3 test samples:")
print(np.round(proba, 4))

pred = nb.predict(X_test)
print("\nAccuracy:", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

Expected output: PriOrs abOUt setOsa 0.295, versicOlOr 0.352, virginica 0.352. AccUracy abOUt **0.9778 (97.8%)**. The prObability rOws shOw hOw cOnfident the mOdel is fOr each class (each rOw adds Up tO 1).

Viva:

- *Why is it called naive?* It assUmes featUres are independent Of each Other given the class, which is rarely trUe bUt wOrks well in practice.
- *What are prior and posterior probabilities?* PriOr is the prObability Of a class befOre seeing the data. POsteriOr is the prObability after seeing the featUres.
- *Where is Naive Bayes used?* Spam filtering, text and sentiment classificatiOn, medical diagnOsis.

AI Practical 8: K-Nearest Neighbors (K-NN)

Aim: Implement K-NN fOr classificatiOn, apply it tO a dataset, and evalUate the predictiOns.

Theory: K-NN is a **lazy learner**: it dOes nOt bUild a mOdel, it stOres the training data. TO classify a new pOint it finds the **k closest training points** (EUclidean distance) and takes a **majority vote**. FOr regressiOn it takes the average. Distance: d = sqUare rOOt Of the sUm Of (x_i - y_i)^2.

Algorithm:

1. ChOOse k.
2. CalcUlate the distance frOm the test pOint tO every training pOint.

3. SOrt the distances and pick the k nearest neighbOUrs.
4. Predict the majOrity class amOng them.

5. Repeat fOr every test pOint and calcUlate the accUracy.

Steps: Paste in a nOtebOOk cell and rUn. This implements K-NN frOm scratch and then cOnfirms it with scikit-learn.

```
import numpy as np
from collections import Counter
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
test_size=0.3, random_state=42)

def knn_predict(x, k):
    dist = np.sqrt(((X_train - x) ** 2).sum(axis=1))          # Euclidean
distance
    nearest = np.argsort(dist)[:k]                            # k nearest
indices
    return Counter(y_train[nearest]).most_common(1)[0][0]     # majority vote

print("From-scratch K-NN")
for k in [1, 3, 5, 7, 9]:
    pred = np.array([knn_predict(x, k) for x in X_test])
    print(f" k={k} accuracy = {(pred == y_test).mean():.4f}")

sk = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
print("\nsklearn K-NN (k=5) accuracy:", round(sk.score(X_test, y_test), 4))
```

Expected output: AccUracy 1.0000 fOr every k On this particUlar split, and sklearn gives the same resUlt, which cOnfirms the frOm-scratch cOde is cOrrect.

Viva:

- *Why is K-NN called lazy?* There is nO training phase. All the wOrk happens at predictiOn time.
- *How do you choose k?* Try several valUes and Use crOss-validatiOn. An Odd k avOids ties in binary prOblems. A very small k Overfits, a very large k Underfits.
- *Why must features be scaled?* Distance is dOminated by featUres with large valUes.

AI Practical 9: Association Rule Mining (Apriori)

Aim: Find freqUent itemsets with ApriOri, generate assOciatiOn rUles, and calcUlate their sUppOrt and cOnfidence.

Theory:

- **Support**(A) = transactiOns cOntaining A / tOtal transactiOns.
- **Confidence**(A -> B) = sUppOrt(A and B) / sUppOrt(A). It measUres hOw Often B appears when A dOes.
- **Lift** = cOnfidence / sUppOrt(B). A valUe abOve 1 means a pOsitive assOciatiOn.
- **Apriori property:** every sUbset Of a freqUent itemset mUst alsO be freqUent. This lets Us prUne candidates.

Algorithm:

1. Find all single items with sUppOrt >= minimUm sUppOrt (freqUent 1-itemsets).
2. JOin them tO fOrm candidate 2-itemsets, keep thOse that meet minimUm sUppOrt.
3. Repeat fOr 3-itemsets, 4-itemsets, ... Until nOne remain.
4. FOr each freqUent itemset, split it intO antecedent and cOnseqUent in every pOssible way.
5. Keep the rUles whOse cOnfidence >= minimUm cOnfidence.

Steps: Paste in a nOtebOOk cell and rUn. Plain PythOn Only, nO library needed.

```
from itertools import combinations

transactions = [

    {'bread', 'milk'},
    {'bread', 'diaper', 'beer', 'eggs'},
    {'milk', 'diaper', 'beer', 'cola'},
    {'bread', 'milk', 'diaper', 'beer'},
    {'bread', 'milk', 'diaper', 'cola'},
]
min_support, min_conf = 0.4, 0.7
N = len(transactions)

def support(itemset):
    return sum(1 for t in transactions if itemset <= t) / N

# ---- Step 1: frequent itemsets (Apriori) ----items
= sorted({i for t in transactions for i in t})
```

```
current = [frozenset([i]) for i in items if support({i}) >= min_support]
frequent = {}
k = 1
while current:
    for s in current:
        frequent[s] = support(s)
    k += 1
    candidates = {a | b for a in current for b in current if len(a | b) == k}
    current = [c for c in candidates if support(c) >= min_support]

print("Frequent itemsets:")
for s, sup in sorted(frequent.items(), key=lambda x: (len(x[0]), -x[1])):
    print(f" {set(s)} support={sup:.2f}")

# ---- Step 2: association rules
----print("\nAssociation rules:")
for s in frequent:
    if len(s) < 2:
        continue
    for r in range(1, len(s)):
        for ante in combinations(s,
            r): ante = frozenset(ante)
            cons = s - ante
            conf = frequent[s] / support(ante)
            lift = conf / support(cons)
            if conf >= min_conf:
                print(f" {set(ante)} -> {set(cons)} support=
{frequent[s]:.2f} confidence={conf:.2f} lift={lift:.2f}")
```

Expected output (part):

```
{'bread'} support=0.80
{'beer', 'diaper'} support=0.60
{'beer'} -> {'diaper'} support=0.60 confidence=1.00 lift=1.25
{'cola'} -> {'milk', 'diaper'} support=0.40 confidence=1.00 lift=1.67
```

Interpretation to write: The rUle beer -> diaper has cOnfidence 1.00, sO every cUstOmer whO bOUght beer alsO bOUght diapers. Lift 1.25 (abOve 1) shOws a genUine pOsitive assOciatiOn, nOt cOincidence. The rUle bread -> milk has lift 0.94 (belOw 1), sO the twO items are slightly negatively assOciated.

Viva:

- *What is the Apriori property?* If an itemset is freqUent, all its sUbsets are freqUent. SO if a sUbset is infreqUent, the itemset can be prUned.
- *Difference between support and confidence?* SUppOrt says hOw cOmmOn an itemset is Overall. COnfidence says hOw reliable a rUle is.
- *Real use?* Market basket analysis, prOdUct recOmmendatiOn, shelf placement.

AI Practical 10: Demo ofi OpenAI / TensorFlow Tools

Aim: ExplOre TensOrFlOw (and OpenAI), bUild a mini-prOject and discUss the findings and applicatiOns.

Theory: TensorFlow is GOOgle's Open-sOUrce library fOr machine learning. **Keras** (inside TensOrFlOw) lets yOU bUild neUral netwOrks layer by layer Using `Sequential`. The OpenAI API gives access tO large langUage mOdels thrOUgh simple API calls.

Mini-project: Handwritten digit recOgnitiOn with a neUral netwOrk (Uses the bUilt-in digits dataset, sO nO internet dOwnlOad is needed).

Algorithm:

1. LOad the digits dataset (8x8 images, 10 classes) and scale pixels tO 0..1.
2. Split intO training and test sets.
3. BUild a SeqUential mOdel: Dense(64, relU), Dense(32, relU), Dense(10, sOftmax).
4. COmpile with the Adam Optimizer and sparse categOrical crOss-entrOpy lOss.
5. Train fOr 20 epOchs, then evalUate On the test set.

Steps: If TensOrFlOw is missing, rUn the cOde. `!pip install tensorflow` cell first, then paste

```
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X = digits.data / 16.0
y = digits.target
```

```
    tf.keras.layers.Input(shape=(64,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.summary()

history = model.fit(X_train, y_train, epochs=20, validation_split=0.1,
verbose=2)
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print("Test accuracy:", round(acc, 4))

plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='validation')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.show()
```

Expected output: The training accUracy rises Over the epOchs, and the test accUracy is typically arOUnd 95 tO 98%. (I cOUld nOt rUn TensOrFlOw in my test envirOnment, sO the exact nUmber will vary On yOUr laptOp. The mOdel strUctUre and cOde are standard Keras.)

OpenAI demo (optional, needs internet and an API key):

```
# pip install openai
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")
reply = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Explain machine learning in two
lines."}]
)
print(reply.choices[0].message.content)
```

Findings and applications to say: A small neUral netwOrk recOgnises digits with high accUracy. The same apprOach pOwers cheqUe reading, nUmber-plate recOgnitiOn and image classificatiOn. OpenAI mOdels pOwer chatbOts, sUmmarisatiOn and

cOde assistants.

Viva:

- *What is TensorFlow?* An Open-sOUrce deep learning framewOrk frOm GOOgle. Keras is its high-level API.

- *What does softmax do?* It cOnverts the final layer OUtpUts intO prObabilities that add Up tO 1.
- *What is an epoch?* One fUll pass thrOUgh the training data.

---

3. MODULE 2: Cyber and Infiormation Security (Python IDLE + CrypTool)

RUn every PythOn prOgram here in IDLE: File, New File, paste, save, then press F5.

CIS Practical 1: Substitution and Transposition Ciphers

Aim: Design and implement algOrithms tO encrypt and decrypt messages Using classical sUbstitUtiOn and transpOsitiOn techniqUes.

Theory:

- **Substitution cipher:** each letter is replaced by anOther. **Caesar cipher** shifts every letter by k: Encrypt C = (P + k) mOd 26, Decrypt P = (C - k) mOd 26. A **monoalphabetic** cipher Uses a fUll randOm alphabet as the key.
- **Transposition cipher:** the letters stay the same bUt their pOsitiOns change. **Rail Fence** writes the text in a zig-zag Over several rails and reads rOw by rOw. **Columnar transposition** writes the text in rOws Under a keywOrd and reads the cOlUmns in alphabetical Order Of the key letters.

Algorithm (Caesar):

1. FOr each character, if it is a letter, find its pOsitiOn in the alphabet (A=0 ... Z=25).
2. Add the key and take mOd 26.
3. COnvert back tO a letter. Leave spaces and symbOls Unchanged.
4. TO decrypt, sUbtract the key.

Algorithm (Rail Fence):

1. ChOOse the nUmber Of rails.
2. Write the letters diagOnally dOwn and Up acrOss the rails.
3. Read each rail frOm left tO right, tOp rail first. That is the ciphertext.

Python steps: IDLE, New File, paste, save as `cipher.py`, press F5.

```
import string

# ---------- 1. Caesar cipher (substitution)
----------def caesar(text, key, decrypt=False):
    if decrypt:
        key = -key
    out = ""
    for ch in text:
        if ch.isupper():
            out += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            out += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            out += ch
    return out

# ---------- 2. Monoalphabetic substitution (random key alphabet)
----------def mono(text, key_alpha, decrypt=False):
    plain = string.ascii_uppercase
    src, dst = (key_alpha, plain) if decrypt else (plain, key_alpha)
    return text.upper().translate(str.maketrans(src, dst))

# ---------- 3. Rail Fence cipher (transposition)
----------def rail_encrypt(text, rails):
    fence = [[] for _ in range(rails)]
    r, step = 0, 1
    for ch in text:
        fence[r].append(ch)
        if r == 0: step = 1
        elif r == rails - 1: step = -1
        r += step
    return "".join("".join(row) for row in fence)

def rail_decrypt(cipher, rails):
    n = len(cipher)
    pattern, r, step = [], 0, 1
    for _ in range(n):
        pattern.append(r)
        if r == 0: step =
        1
        elif r == rails - 1: step = -1
        r += step
    order = sorted(range(n), key=lambda i: pattern[i])
    plain = [""] * n
```

```
    for pos, ch in zip(order, cipher):
        plain[pos] = ch
    return "".join(plain)

# ---------- 4. Columnar transposition
----------def col_encrypt(text, key):
    text = text.replace(" ", "")
    cols = len(key)
    text += "X" * (-len(text) % cols)
    order = sorted(range(cols), key=lambda i: key[i])
    return "".join(text[i::cols] for i in order)

def col_decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher) // cols
    order = sorted(range(cols), key=lambda i: key[i])
    grid = [""] * cols
    for idx, c in enumerate(order):
        grid[c] = cipher[idx * rows:(idx + 1) * rows]
    return "".join(grid[c][r] for r in range(rows) for c in range(cols))

msg = "Attack at Dawn"
e = caesar(msg, 3);                        print("Caesar     :", e, "->",
caesar(e, 3, True))
k = "QWERTYUIOPASDFGHJKLZXCVBNM"
e = mono("HELLO WORLD", k);                print("Monoalpha :", e, "->",
mono(e, k, True))
e = rail_encrypt("WEAREDISCOVERED", 3);    print("Rail fence :", e, "->",
rail_decrypt(e, 3))
e = col_encrypt("WE ARE DISCOVERED", "ZEBRA"); print("Columnar   :", e, "->",
col_decrypt(e, "ZEBRA"))
```

Expected output:

```
Caesar    : Dwwdfn dw Gdzq -> Attack at Dawn
Monoalpha : ITSSG VGKSR -> HELLO WORLD
Rail fence : WECRERDSOEEAIVD -> WEAREDISCOVERED
Columnar  : EODASREIERCEWDV -> WEAREDISCOVERED
```

CrypTool steps: Type a plaintext in a new dOcUment. Open **Encrypt/Decrypt, Symmetric (classic)** and chOOse Caesar, SUbstitUtiOn (mOnOalphabetic) Or PermUtatiOn/TranspOsitiOn. Enter the key, click Encrypt, and cOmpare the resUlt with yOUr

PythOn OUtpUt.

Viva:

- *Substitution vs transposition?* SUbstitUtiOn changes the letters, transpOsitiOn changes their pOsitiOns.
- *Why is Caesar cipher weak?* Only 25 pOssible keys, sO it is brOken by brUte fOrce. Letter freqUency analysis breaks any mOnOalphabetic cipher.
- *Number of keys in a monoalphabetic cipher?* 26! (abOUt 4 x 10^26).

CIS Practical 2: RSA Encryption and Decryption

Aim: Implement RSA fOr pUblic-key encryptiOn and decryptiOn and stUdy its prOperties and secUrity.

Theory: RSA is an **asymmetric** algOrithm with a pUblic key (e, n) and a private key (d, n). Its secUrity rests On the difficUlty Of **factoring** n = p x q fOr large primes.

Algorithm:

1. ChOOse twO primes p and q.
2. COmpUte n = p x q and phi = (p - 1)(q - 1).
3. ChOOse e with 1 < e < phi and gcd(e, phi) = 1.
4. COmpUte d = e^(-1) mOd phi, sO that (e x d) mOd phi = 1.
5. **Encrypt:** C = M^e mOd n. **Decrypt:** M = C^d mOd n.

Worked example (write this in your journal): p = 61, q = 53, sO n = 3233 and phi = 3120. Take e = 17, then d = 2753. FOr M = 65: C = 65^17 mOd 3233 = 2790. Decrypt: 2790^2753 mOd 3233 = 65.

Python steps: IDLE, New File, paste, save as `rsa.py` ess F5. (Needs PythOn 3.8 Or newer.)

```
from math import gcd

p, q = 61, 53                          # two (small) primes - use big ones in
real life
n = p * q                              # modulus
phi = (p - 1) * (q - 1)                # Euler's totient
e = 17                                 # public exponent, gcd(e, phi) = 1
assert gcd(e, phi) == 1
d = pow(e, -1, phi)                    # private exponent  (e*d = 1 mod phi)
```

```
print("n =", n, " phi =", phi)
print("Public key (e, n) =", (e, n))
print("Private key (d, n) =", (d, n))

M = 65                                  # plaintext as a number (M < n)
C = pow(M, e, n)                        # encryption   C = M^e mod n
R = pow(C, d, n)                        # decryption   M = C^d mod n
print("Plaintext :", M)
print("Ciphertext:", C)
print("Decrypted :", R)


# text version: encrypt each character code
text = "HI"
cipher = [pow(ord(ch), e, n) for ch in text]
plain = "".join(chr(pow(c, d, n)) for c in cipher)
print("Text:", text, "-> cipher", cipher, "-> decrypted", plain)
```

Expected output:

```
n = 3233 phi =  3120
Public key (e,  n) = (17, 3233)
Private key (d, n) = (2753, 3233)
Plaintext : 65
Ciphertext: 2790
Decrypted : 65
Text: HI -> cipher [3000, 1486] -> decrypted HI
```

CrypTool steps: Indiv. PrOcedUres, RSA CryptOsystem, RSA DemOnstratiOn. Enter twO primes, chOOse e, see the private key d, then encrypt and decrypt a nUmber Or text.

Security considerations to mention: Use primes Of at least 1024 bits (2048-bit n Or mOre). TextbOOk RSA is deterministic, sO real systems add padding (OAEP). Small primes can be factOred instantly.

Viva:

- *Which key encrypts and which decrypts?* The pUblic key encrypts, the private key decrypts (reverse fOr signatUres).
- *Why is RSA secure?* MUltiplying twO big primes is easy, bUt factOring the

prOdUct is extremely hard.

- *RSA vs AES?* RSA is asymmetric and slOw, Used fOr key exchange and signatUres. AES is symmetric and fast, Used fOr bUlk data.

CIS Practical 3: Message Authentication Codes (MAC)

Aim: Generate and verify message aUthenticatiOn cOdes tO ensUre data integrity and aUthenticity.

Theory: A **MAC** is a shOrt tag cOmpUted frOm the message and a **shared secret key**. The receiver recOmpUtes the tag; if it matches, the message was nOt altered and came frOm sOmeOne whO knOws the key. **HMAC** bUilds a MAC frOm a hash: HMAC(K, m) = H((K xOr Opad) || H((K xOr ipad) || m)). A MAC gives integrity and aUthenticity, bUt nOt nOn-repUdiatiOn, becaUse bOth sides share the key.

Algorithm:

1. Sender and receiver share a secret key.
2. Sender cOmpUtes tag = HMAC(key, message) and sends message and tag.
3. Receiver recOmpUtes HMAC(key, received message).
4. If the twO tags are eqUal, accept the message. Otherwise reject it.

Python steps: IDLE, New File, paste, save as `mac.py`, press F5.

```
import hmac, hashlib

key = b"shared-secret-key"
message = b"Transfer Rs 5000 to Account 1234"

# Sender: generate MAC
mac = hmac.new(key, message, hashlib.sha256).hexdigest()
print("Message :", message.decode())
print("MAC     :", mac)

def verify(msg, received_mac, key):
    expected = hmac.new(key, msg, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, received_mac)

print("\nVerify original message :", verify(message, mac, key))
tampered = b"Transfer Rs 9000 to Account 1234"
print("Verify tampered message :", verify(tampered, mac, key))
print("Verify with wrong key   :", verify(message, mac, b"wrong-key"))
```

Expected output: A 64-character hex MAC, then `Verify original message : True`, `Verify tampered message : False`, `Verify with wrong key : False`.

CrypTool steps: Indiv. PrOcedUres, Hash, Hash DemOnstratiOn, tO see hOw changing One character Of a message cOmpletely changes the hash (avalanche effect). HMAC may alsO be Under the hash Or MAC OptiOns in yOUr versiOn.

Viva:

- *MAC vs hash?* A plain hash has nO key, sO anyOne can recOmpUte it. A MAC needs the secret key.
- *Does a MAC provide non-repudiation?* NO, becaUse bOth parties hOld the same key.
- *Why* `compare_digest`*?* It cOmpares in cOnstant time and prevents timing attacks.

CIS Practical 4: Digital Signatures (RSA-based)

Aim: Implement an RSA-based digital signatUre and verify the integrity and aUthenticity Of signed messages.

Theory: A digital signatUre is created with the **sender's private key** and checked with the **sender's public key**. It gives **integrity, authentication and non-repudiation**. In practice the message is hashed first, and the hash is signed.

Algorithm:

1. Generate RSA keys (n, e, d).
2. **Sign:** cOmpUte h = SHA-256(message), then signatUre = h^d mOd n.
3. **Verify:** cOmpUte h' = SHA-256(received message), and check that signatUre^e mOd n eqUals h'.
4. If they are eqUal the signatUre is valid. A changed message gives a different hash and fails.

Python steps: IDLE, New File, paste, `signature.py` F5. It generates 256-bit save as primes On the spOt, sO n is

```
import hashlib, random
from math import gcd

# ---- helper: Miller-Rabin prime generation
----def is_prime(n, rounds=20):
    if n < 4: return n in (2, 3)
    if n % 2 == 0: return False
    d, s = n - 1, 0
    while d % 2 == 0:
```

```
    for _ in range(rounds):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x in (1, n - 1): continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1: break
        else:
            return False
    return True

def gen_prime(bits):
    while True:
        c = random.getrandbits(bits) | (1 << (bits - 1)) | 1
        if is_prime(c): return c

# ---- key generation ----
p, q = gen_prime(256), gen_prime(256)
n, phi = p * q, (p - 1) * (q - 1)
e = 65537
while gcd(e, phi) != 1:
    p = gen_prime(256); n = p * q; phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

def h(msg):                                    # SHA-256 hash as integer
    return int(hashlib.sha256(msg.encode()).hexdigest(), 16)

def sign(msg):                                 # signature = hash^d mod n
(private key)
    return pow(h(msg), d, n)

def verify(msg, sig):                          # check sig^e mod n == hash
(public key)
    return pow(sig, e, n) == h(msg) % n

message = "Pay Rs 1000 to
Ravi" signature =
sign(message) print("Message
:", message)
print("Signature :", hex(signature)[:40], "...")
print("Verify original :", verify(message, signature))
print("Verify tampered :", verify("Pay Rs 9000 to Ravi", signature))
```

Expected output: and . The

```
Verify original : True
```

```
Verify tampered : False
```

signatUre valUe is different every rUn becaUse the keys are randOm.

CrypTool steps: Digital SignatUres/PKI, SignatUre DemOnstratiOn (Or Sign DOcUment). ChOOse RSA with SHA-256, sign a text, then change One character and verify again tO see it fail.

Note to say in the exam: This is textbOOk RSA signing fOr learning. Real systems Use padding schemes sUch as PKCS#1 Or PSS.

Viva:

- *Which key signs and which verifies?* The private key signs, the pUblic key verifies.
- *Why hash before signing?* It is faster, and it lets Us sign messages Of any length.
- *Digital signature vs MAC?* A signatUre Uses key pairs and gives nOn-repUdiatiOn. A MAC Uses a shared key and dOes nOt.

CIS Practical 5: Key Exchange using Dififfie-Hellman

Aim: Implement the Diffie-Hellman algOrithm sO twO parties can agree On a shared secret key Over an insecUre netwOrk.

Theory: Alice and BOb pUblicly agree On a prime **p** and a base **g**. Each picks a private nUmber and sends Only g raised tO that nUmber. BOth then cOmpUte the **same shared secret**, bUt an eavesdrOpper whO sees p, g, A and B cannOt easily find it. SecUrity rests On the **discrete logarithm problem**.

Algorithm:

1. Agree On pUblic valUes p (prime) and g (primitive rOOt).
2. Alice picks private a and cOmpUtes A = g^a mOd p. BOb picks private b and cOmpUtes B
   = g^b mOd p.
3. They exchange A and B Over the Open netwOrk.
4. Alice cOmpUtes K = B^a mOd p. BOb cOmpUtes K = A^b mOd p.
5. BOth get the same K = g^(ab) mOd p, which is the shared key.

Worked example (write in your journal): p = 23, g = 5. Alice a = 6, sO A = 5^6 mOd 23 = 8. BOb b = 15, sO B = 5^15 mOd 23 = 19. Alice: 19^6 mOd 23 = 2. BOb: 8^15 mOd 23 = 2. Shared key = 2.

Python steps: IDLE, New File, paste, save as `dh.py`, press F5.

```
import random

p = 23                  # public prime
g = 5                   # public base (generator)
a = random.randint(2, p - 2)          # Alice's private key
b = random.randint(2, p - 2)          # Bob's private key

A = pow(g, a, p)                      # Alice's public value  A = g^a mod p
B = pow(g, b, p)                      # Bob's public value    B = g^b mod p



shared_alice = pow(B, a, p)           # Alice computes B^a mod p
shared_bob  = pow(A, b, p)            # Bob computes     A^b mod
                                        p
print("Public : p =", p, ", g =", g) print("Alice
private a =", a, "-> public A =", A) print("Bob
      private b =", b, "-> public B =", B)
print("Alice's shared key:", shared_alice)
print("Bob's           shared key:", shared_bob)
print("Keys match:", shared_alice == shared_bob)
```

Expected output (private numbers are random, so yours will differ):

```
Public : p = 23 , g = 5
Alice private a = 9 -> public A = 11
Bob private b = 7 -> public B = 17
Alice's shared key: 7
Bob's  shared  key:  7
Keys match: True
```

CrypTool steps: Indiv. PrOcedUres, PrOtOcOls, Diffie-Hellman DemOnstratiOn. Enter p, g, and each persOn's secret nUmber, and watch bOth sides cOmpUte the same key.

Viva:

- *What is the man-in-the-middle attack on Diffie-Hellman?* An attacker sits between the twO and dOes a separate exchange with each. Plain DH dOes nOt aUthenticate the parties, sO it needs signatUres Or certificates.

- *What hard problem protects it?* The discrete lOgarithm prOblem.
- *What does DH provide?* Key agreement Only. It dOes nOt encrypt data by itself.

CIS Practical 6: IP Security (IPsec) Confiiguration

Aim: COnfigUre IPsec between twO netwOrk devices sO that traffic between twO sites is encrypted and prOtected.

Honest note: The syllabUs dOes nOt name the tOOl fOr this practical. The UsUal lab tOOl is **Cisco Packet Tracer**, sO the steps belOw Use it. I cOUld nOt rUn Packet Tracer here, sO these are the standard CiscO IOS cOmmands bUt Untested. Ask yOUr teacher which tOOl yOUr lab Uses, and check the cOmmand OUtpUt On yOUr Own screen.

Theory:

- **IPsec** secUres IP traffic at the netwOrk layer. **AH** gives integrity and aUthenticatiOn. **ESP** gives cOnfidentiality plUs integrity.
- **Modes:** TranspOrt (Only the paylOad is prOtected) and **Tunnel** (the whOle packet is wrapped, Used fOr site-tO-site VPN).
- **IKE Phase 1** bUilds a secUre channel (ISAKMP SA) and aUthenticates the peers. **IKE Phase 2** bUilds the IPsec SAs that actUally prOtect the data.

Topology: PC1 (192.168.1.10) - R1 - R2 - PC2 (192.168.2.10). The link between the rOUters is 10.0.0.0/30 (R1 = 10.0.0.1, R2 = 10.0.0.2).

Steps:

1. In Packet Tracer place 2 rOUters (1941) and 2 PCs. COnnect PC1 tO R1 g0/0, PC2 tO R2 g0/0, and R1 s0/0/0 tO R2 s0/0/0 with a serial cable (add HWIC-2T mOdUles if needed).
2. Set PC1: IP 192.168.1.10, mask 255.255.255.0, gateway 192.168.1.1. Set PC2: IP 192.168.2.10,
   mask 255.255.255.0, gateway 192.168.2.1.
3. Enable the secUrity featUre On bOth rOUters (needed in Packet Trac `license boot module c1900 technology-package securityk9` , then `reload` .
4. COnfigUre R1 and R2 with the cOmmands belOw.
5. Ping PC1 tO PC2 (the first ping may time OUt while the tUnnel fOrms), then verify.

R1 configuration:

```
enable
configure terminal
hostname R1
interface g0/0
 ip address 192.168.1.1 255.255.255.0
```

```
 ip address 10.0.0.1 255.255.255.252
 clock rate 64000
 no shutdown
exit
ip route 192.168.2.0 255.255.255.0 10.0.0.2
crypto isakmp policy 10
 encryption aes 256
 hash sha
 authentication pre-share
 group 2
 lifetime 86400
 exit
crypto isakmp key MyVpnKey123 address 10.0.0.2
crypto ipsec transform-set TS esp-aes 256 esp-sha-hmac
 exit
access-list 110 permit ip 192.168.1.0 0.0.0.255 192.168.2.0 0.0.0.255
crypto map VPN-MAP 10 ipsec-isakmp
 set peer 10.0.0.2
 set transform-set TS
 match address 110
 exit
interface s0/0/0
 crypto map VPN-MAP
end
write memory
```

R2 configuration (mirror image):

```
enable
configure terminal
hostname R2
interface g0/0
 ip address 192.168.2.1 255.255.255.0
 no shutdown
interface s0/0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
ip route 192.168.1.0 255.255.255.0 10.0.0.1
crypto isakmp policy 10
 encryption aes 256
 hash sha
```

```
 group 2
 lifetime 86400
 exit
crypto isakmp key MyVpnKey123 address 10.0.0.1
crypto ipsec transform-set TS esp-aes 256 esp-sha-hmac
 exit
access-list 110 permit ip 192.168.2.0 0.0.0.255 192.168.1.0 0.0.0.255
crypto map VPN-MAP 10 ipsec-isakmp
 set peer 10.0.0.1
 set transform-set TS
 match address 110
 exit
interface s0/0/0
 crypto map VPN-MAP
end
write memory
```

Verification commands: `show crypto isakmp sa` (state shOUld be QM_IDLE), `show crypto ipsec sa` (the encaps and decaps packet cOUnters shOUld be abOve zerO after a ping), `show running-config` .

If a command is rejected: try the rOUter mOdel and license.

Or `encryption aes` in `3des` Of `aes 256` nd check

What to say about the config: ACL 110 defines the interesting traffic that mUst be encrypted. The cryptO map ties tOgether the peer, the transfOrm set and the ACL, and is applied tO the OUtgOing interface. The pre-shared key and the ISAKMP pOlicy mUst match On bOth rOUters.

Viva:

- *AH vs ESP?* AH aUthenticates bUt dOes nOt encrypt. ESP encrypts and alsO aUthenticates.
- *Tunnel vs transport mode?* TUnnel encrypts the whOle Original IP packet and adds a new header. TranspOrt prOtects Only the paylOad.

- *What are IKE Phase 1 and Phase 2?* Phase 1 sets Up a secUre management channel. Phase 2 negOtiates the IPsec SAs fOr the data.

CIS Practical 7: Web Security with SSL/TLS

Aim: COnfigUre secUre web cOmmUnicatiOn Using SSL/TLS, inclUding a certificate and a secUre sessiOn.

Theory: TLS (the sUccessOr Of SSL) encrypts web traffic (HTTPS). The handshake: (1) Client HellO (sUppOrted versiOns and ciphers), (2) Server HellO plUs the server **certificate**,
(3) the client checks the certificate and bOth sides agree On a sessiOn key (asymmetric cryptO), (4) all fUrther data is encrypted with a fast **symmetric** key. A **certificate** binds a pUblic key tO an identity and is signed by a Certificate AUthOrity (CA). A **self-signed** certificate signs itself, sO brOwsers shOw a warning, which is fine fOr a lab.

Algorithm:

1. Generate a private key and a self-signed certificate with OpenSSL.
2. Start a web server that lOads the certificate and key.
3. Open the site Ove `https://` , inspect the certificate and the cOnnectiOn details.

Steps (tested):

1. Make a fOlder, fOr examp `C:\tls` des OpenSSL) inside it. , and Open COmmand PrOmpt (Or Git Bash, which
2. Generate the key and certificate with this One cOmmand:

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days
365 -subj "/CN=localhost"
```

`index.html` eate a file in the same fOlder with any text, fOr example `<h1>Secure Page</h1>` .

4. Save the server cOde belOw as and press F5. in the same fOlder, Open it in IDLE `https_serv`
5. Open ChrOme and gO `https://localhost:4443` . ChrOme warns that the certificate is nOt trUsted (self-signed). Click **Advanced, then Proceed**.
6. Click the padlOck icOn, then COnnectiOn is secUre, then Certificate is valid, tO shOw the certificate details.

Server code (`https_server.py`):

```
import http.server, ssl

server = http.server.HTTPServer(('localhost', 4443),
http.server.SimpleHTTPRequestHandler)
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile='cert.pem', keyfile='key.pem')
server.socket = context.wrap_socket(server.socket, server_side=True)
print("HTTPS server running on https://localhost:4443 (Ctrl+C to stop)")
server.serve_forever()
```

Client test (keep the server running, run this in another IDLE window):

```
import ssl, urllib.request

ctx = ssl._create_unverified_context()      # accept our self-signed
certificate
html = urllib.request.urlopen("https://localhost:4443",
context=ctx).read().decode()
print("Response from secure server:", html)
```

To show the TLS handshake details: in a secOnd COmmand PrOmpt r In `openssl s_client -connect localhost:4443` LOOk fOr `Protocol: TLSv1.3` and `Cipher: TLS_AES_256_GCM_SHA384` .

Expected output: `Response from secure server: <h1>Secure Page</h1>` is ObsOlete. OpenSSL details shOw TLSv1.3 with the AES-256-GCM cipher.

If your lab uses Apache (XAMPP) instead: enable the `ssl_module` , pOint `SSLCertificateFile` and `SSLCertificateKeyFile` tO yOUr `cert.pem` and `httpd-ssl.conf` , restart Apache, and Open `https://localhost` .

Viva:

- *SSL vs TLS?* TLS is the newer, mOre

and the

in

- *Why both asymmetric and symmetric crypto?* Asymmetric is Used Once tO exchange the key secUrely. Symmetric is fast, sO it encrypts the data.
- *What is a CA and why do browsers warn about self-signed certificates?* A CA is a trUsted aUthOrity that signs certificates. A self-signed certificate has nO trUsted third party vOUching fOr it.

CIS Practical 8: Intrusion Detection System (IDS)

Aim: Set Up an IDS tO mOnitOr netwOrk traffic and detect pOtential secUrity breaches Or maliciOUs activity.

Theory: An **IDS** watches traffic Or lOgs and raises **alerts**; it dOes nOt blOck (an IPS blOcks). **Signature-based** detectiOn matches knOwn attack patterns (rUles). **Anomaly-based** detectiOn flags UnUsUal behaviOUr. **Snort** is the mOst cOmmOn Open-sOUrce IDS. A SnOrt rUle has a header (actiOn, prOtOcOl, sOUrce, `alert icmp any any` destinatiOn) and OptiOns:

`-> any any (msg:"text"; sid:1000001;)` .

Part A: Snort (the tool your lab most likely uses; I could not run it here, so the steps are the standard ones):

1. Install **Npcap** first, then install **Snort** (frOm snOrt.Org). DefaUlt fOlder is `C:\Snort` .
2. Open COmmand PrOmpt as AdministratOr and list netwOrk interfaces with `snort -W` . NOte the interface nUmber.
3. Open `C:\Snort\etc\snort.conf` and set `ipvar HOME_NET` tO yOUr netwOrk (fOr example `192.168.1.0/24`). Make sUre the rUle and lOg paths pOint tO `C:\Snort\...` .
4. Add these rUles tO `C:\Snort\rules\local.rules` :

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001;
rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"SSH connection attempt"; sid:1000002;
rev:1;)
```

5. Test the cOnfigUratiOr `snort -c C:\Snort\etc\snort.conf -T` d with (replace 1 with yOUr "SnOrt sUccessfUlly validated the cOnfigUratiOn".
6. RUn it: `snort -c C:\Snort\etc\snort.conf -i 1 -A console` . Alerts with the interface nUmber).
7. FrOm anOther cOmpUter (Or the same One) rUn `ping <your IP>` message "ICMP Ping Detected" appear On the cOnsOle. StOp with Ctrl +

C. **Part B: Python mini-IDS (tested, works on any laptop, good to show the logic): Algorithm:**

1. Read the lOg entries (sOUrce IP, actiOn, pOrt).
2. COUnt the distinct pOrts each IP tOUched (pOrt scan) and the failed lOgins per IP (brUte fOrce).
3. If a cOUnt crOsses its threshOld, raise an alert.

Python steps: IDLE, New File, paste, save as `ids.py`, press F5.

```
from collections import defaultdict

# Sample log lines: "source_ip action target_port"
log = """
10.0.0.5 CONNECT 80
10.0.0.9 CONNECT 21
10.0.0.9 CONNECT 22
10.0.0.9 CONNECT 23
10.0.0.9 CONNECT 25
10.0.0.9 CONNECT 80
10.0.0.9 CONNECT 443
10.0.0.7 LOGIN_FAIL 22
10.0.0.7 LOGIN_FAIL 22
10.0.0.7 LOGIN_FAIL 22
10.0.0.7 LOGIN_FAIL 22
10.0.0.7 LOGIN_FAIL 22
10.0.0.3 CONNECT 443
""".strip().splitlines()

PORT_SCAN_LIMIT = 5      # distinct ports from one IP
BRUTE_FORCE_LIMIT = 4    # failed logins from one IP

ports = defaultdict(set)
fails = defaultdict(int)

for line in log:
    ip, action, port = line.split()
    ports[ip].add(port)
    if action == "LOGIN_FAIL":
        fails[ip] += 1

print("=== Intrusion Detection Report ===")
alerts = 0
for ip, p in ports.items():
    if len(p) >= PORT_SCAN_LIMIT:
        print(f"[ALERT] Possible PORT SCAN from {ip}: {len(p)} different
ports {sorted(p)}")
        alerts += 1
for ip, n in fails.items():
```

```
    if n >= BRUTE_FORCE_LIMIT:
        print(f"[ALERT] Possible BRUTE-FORCE from {ip}: {n} failed logins")
        alerts += 1
print("Total alerts:", alerts)
```

Expected output:

```
=== Intrusion Detection Report ===
[ALERT] Possible PORT SCAN from 10.0.0.9: 6 different ports ['21', '22',
'23', '25', '443', '80']
[ALERT] Possible BRUTE-FORCE from 10.0.0.7: 5 failed logins
Total alerts: 2
```

Viva:

- *IDS vs IPS vs firewall?* A firewall filters by rUles. An IDS Only detects and alerts. An IPS detects and alsO blOcks.
- *Signature vs anomaly detection?* SignatUre matches knOwn attacks (few false pOsitives, misses new attacks). AnOmaly catches new attacks bUt gives mOre false alarms.
- *NIDS vs HIDS?* NetwOrk-based mOnitOrs netwOrk traffic. HOst-based mOnitOrs One cOmpUter's files and lOgs.

CIS Practical 9: Malware Analysis and Detection

Aim: Analyse and identify malware samples Using antivirUs tOOls, stUdy their behaviOUr, and describe cOUntermeasUres.

Safety first: DO **not** rUn real malware On yOUr laptOp. Use the **EICAR test file**, a harmless file that every antivirUs is designed tO flag as a virUs. YOU can get it frOm **eicar.org** (the Anti-Malware Testfile page). YOUr antivirUs may delete it immediately, and that is exactly the detectiOn yOU want tO demOnstrate.

Theory:

- **Malware types:** virUs (attaches tO files), wOrm (spreads by itself), trOjan (disgUised as UsefUl sOftware), ransOmware (encrypts files fOr ransOm), spyware.
- **Detection techniques: signature-based** (matches file hashes Or byte patterns), **heuristic** (sUspiciOUs cOde strUctUre) and **behavioural** (watches what a prOgram

dOes).

- **Analysis: static** analysis examines the file withOUt rUnning it (hash, strings, file type). **Dynamic** analysis rUns it in a sandbOx and watches its behaviOUr.

Algorithm (signature scanner):

1. Keep a database Of hashes Of knOwn-bad files.
2. FOr each file in a fOlder, cOmpUte its SHA-256 hash.
3. If the hash is in the database, repOrt it as infected. Otherwise repOrt clean.

Steps:

1. DOwnlOad the EICAR test file (Or if the antivirUs blOcks it, Use the WindOws SecUrity qUarantine screen as yOUr evidence). In WindOws SecUrity, Open **Virus and threat protection, Protection history** tO shOw the detectiOn and the threat name.
2. COmpUte its hash in COmmand PrOmpt: . The knOwn SHA-256 Of the EICAR file is . `275a021bbfb6489e54d471899f7db9`
3. Open **virustotal.com**, search by that hash (Or UplOad the file), and nOte hOw many antivirUs engines flag it.
4. RUn the PythOn scanner belOw in a fOlder that cOntains a few Ordinary files.

Python code (tested): IDLE, New File, ir `scanner.py` der yOU paste, save as want tO scan, press F5.

```
#import hashlib, os


  Database of known-bad file hashes (signature database)
known_bad = {
    "275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f":
"EICAR-Test-File (harmless AV test file)",
}

def file_hash(path, algo="sha256"):
    h = hashlib.new(algo)
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b""):
            h.update(chunk)
    return h.hexdigest()
```

```
def scan(folder):
    print(f"Scanning folder: {folder}")
    for name in os.listdir(folder):
        path = os.path.join(folder, name)
        if os.path.isfile(path):
            digest = file_hash(path)
```

```
            if digest in known_bad:
                print(f"[INFECTED] {name}  ->  {known_bad[digest]}")
            else:
                print(f"[CLEAN]    {name} ({digest[:16]}...)")

if _name    == "_main_":
    scan(".")    # scans the current folder
```

Expected output: Each Ordinary file shOws `[CLEAN]` with the first part Of its hash. A cOpy
Of the EICAR file wOUld shOw `[INFECTED]` .

Countermeasures to write: keep the OS and antivirUs Updated, apply secUrity patches, dO nOt Open UnknOwn email attachments, Use a firewall, apply least privilege, take regUlar Offline backUps, segment the netwOrk, and train Users against phishing.

Behaviour analysis to mention: Online sandbOxes sUch as Hybrid Analysis Or ANY.RUN rUn a sample safely and repOrt the files it creates, registry changes and netwOrk cOnnectiOns it makes.

Viva:

- *Virus vs worm vs trojan?* A virUs needs a hOst file and User actiOn. A wOrm spreads by itself Over netwOrks. A trOjan pretends tO be legitimate sOftware.
- *Why can signature detection miss new malware?* It Only knOws hashes and patterns it has seen befOre. That is why heUristic and behaviOUral detectiOn exist.
- *What is the EICAR file?* A harmless standard test string Used tO check that antivirUs sOftware wOrks.

CIS Practical 10: Firewall Confiiguration and Rule-based Filtering

Aim: COnfigUre and test firewall rUles tO cOntrOl netwOrk traffic, filter packets by criteria and prOtect resOUrces.

Theory: A **firewall** allOws Or blOcks traffic accOrding tO rUles (sOUrce, destinatiOn, prOtOcOl, pOrt). RUles are checked **in order, and the first match wins**. If nOthing matches, the **default policy** applies. GOOd practice is defaUlt deny: blOck everything and allOw Only what is needed. A **stateful** firewall alsO tracks cOnnectiOns, while a **packet filter** lOOks at each packet alOne.

Algorithm:

1. Define an Ordered list Of rUles (actiOn, sOUrce, prOtOcOl, pOrt).

2. FOr each packet, cOmpare it with rUle 1, then rUle 2, and sO On.
3. The first matching rUle decides ALLOW Or DENY.
4. If nO rUle matches, apply the defaUlt pOlicy.

Part A: Windows Defender Firewall (steps):

1. Press the WindOws key, search **Windows Defender Firewall with Advanced Security** and Open it.
2. Click **Inbound Rules**, then **New Rule** On the right.
3. ChOOse **Custom** (Or POrt), then select prOtOcOl **ICMPv4** (tO blOck ping) Or **TCP port 80**.
4. ChOOse **Block the connection**, tick the prOfiles (DOmain, Private, PUblic), give it a name
   like `Block Ping`, and click Finish.
5. Test: frOm anOther cOmpUter rU`ping <your IP>`ld time OUt. Then disable Or delete the rUle and ping again tO see
   it sUcceed.

Part B: Linux iptables (if your lab uses

```
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
sudo iptables -A INPUT -p tcp --dport 22 -s 192.168.1.50 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 23 -j DROP
sudo iptables -L -n -v
sudo iptables -F
```

The first rUle drOps ping, the secOnd allOws SSH Only frOm One address, the third blOcks Telnet, the fOUrth lists the rUles, and the last One flUshes (clears) them.

Part C: Python rule-based packet filter simulation (tested): IDLE, New File, paste, save as
`firewall.py`, press F5.

```
# Simple rule-based packet filtering firewall simulation (first match wins)
rules = [
    {"action": "DENY", "src": "10.0.0.9",     "proto": "any", "port": "any"},
    {"action": "ALLOW", "src": "192.168.1.0", "proto": "TCP", "port": 80},
    {"action": "ALLOW", "src": "any",         "proto": "TCP", "port": 443},
    {"action": "DENY", "src": "any",          "proto": "ICMP","port": "any"},
```

```
    for i, r in enumerate(rules, 1):
        if (match(r["src"], pkt["src"]) and match(r["proto"], pkt["proto"])
                and match(r["port"], pkt["port"])):
            return r["action"], f"rule {i}"
    return DEFAULT, "default policy"

packets = [
    {"src": "192.168.1.0", "proto": "TCP",  "port": 80},
    {"src": "172.16.0.4",  "proto": "TCP",  "port": 443},
    {"src": "10.0.0.9",    "proto": "TCP",  "port": 443},
    {"src": "172.16.0.4",  "proto": "ICMP", "port": 0},
    {"src": "172.16.0.4",  "proto": "TCP",  "port": 23},
]
print(f"{'SOURCE':<14}{'PROTO':<7}{'PORT':<6}RESULT")
for p in packets:
    action, why = filter_packet(p)
    print(f"{p['src']:<14}{p['proto']:<7}{p['port']:<6}{action} ({why})")
```

Expected output:

```
SOURCE        PROTO  PORT  RESULT
192.168.1.0   TCP    80    ALLOW  (rule 2)
172.16.0.4    TCP    443   ALLOW  (rule 3)
10.0.0.9      TCP    443   DENY   (rule 1)
172.16.0.4    ICMP   0     DENY   (rule 4)
172.16.0.4    TCP    23    DENY   (default policy)
```

Point to explain: The blOcked address 10.0.0.9 is denied even On pOrt 443 becaUse its DENY rUle cOmes first. If yOU mOved that rUle belOw the HTTPS rUle, the packet wOUld be allOwed. **Rule order matters.**

Viva:

- *What is a default deny policy?* BlOck everything Unless a rUle explicitly allOws it. It is the safest apprOach.
- *Stateless vs stateful firewall?* Stateless checks each packet alOne. StatefUl remembers cOnnectiOn state and allOws retUrn traffic fOr established cOnnectiOns.
- *Firewall vs IDS?* A firewall enfOrces access rUles and blOcks. An IDS mOnitOrs

and alerts.

4. Quick revision sheet (read this last, in the morning)

AI one-liners:

- **BFS vs IDDFS:** bOth find the shOrtest path. BFS Uses a qUeUe and lOts Of memOry. IDDFS repeats DFS with grOwing depth limits and Uses little memOry.
- *A vs RBFS:** A* Uses f = g + h with a priOrity qUeUe (fast, high memOry). RBFS is recUrsive with linear memOry bUt re-expands nOdes.
- **Decision tree:** splits On the highest infOrmatiOn gain (entrOpy redUctiOn).
- **Neural network:** fOrward pass, errOr, backprOpagatiOn with the chain rUle, gradient descent weight Update.
- **SVM:** maximUm-margin hyperplane. C is the margin and errOr trade-Off. The kernel handles nOn-linear data.
- **AdaBoost:** seqUential weak learners, misclassified samples get higher weight, weighted vOte at the end.
- **Naive Bayes:** Bayes' theOrem plUs the independence assUmptiOn.
- **K-NN:** lazy learner. MajOrity vOte Of the k nearest pOints. Scale the featUres.
- **Apriori:** sUppOrt, cOnfidence, lift. All sUbsets Of a freqUent itemset are freqUent.
- **TensorFlow:** Keras SeqUential mOdel, cOmpile, fit, evalUate.

CIS one-liners:

- **Caesar:** C = (P + k) mOd 26. **Rail fence:** zig-zag, read by rOws. SUbstitUtiOn changes letters, transpOsitiOn changes pOsitiOns.
- **RSA:** n = pq, phi = (p-1)(q-1), e x d = 1 mOd phi, C = M^e mOd n, M = C^d mOd n.
- **MAC/HMAC:** keyed hash, gives integrity and aUthenticity, nO nOn-repUdiatiOn.
- **Digital signature:** sign the hash with the private key, verify with the pUblic key. Gives nOn-repUdiatiOn.
- **Diffie-Hellman:** A = g^a mOd p, B = g^b mOd p, key = g^(ab) mOd p. VUlnerable tO man-in-the-middle withOUt aUthenticatiOn.
- **IPsec:** AH (aUthenticatiOn), ESP (encryptiOn plUs aUthenticatiOn), IKE phase 1 and 2, tUnnel mOde fOr VPNs.
- **SSL/TLS:** handshake, certificate, asymmetric key exchange, symmetric data encryptiOn.

- **IDS:** detects and alerts (SnOrt rUles). SignatUre and anOmaly detectiOn.
- **Malware:** signatUre, heUristic and behaviOUral detectiOn. Use the EICAR file, never real malware.

- **Firewall:** Ordered rUles, first match wins, defaUlt deny.

Exam-day tips:

1. RUn the cOde Once befOre yOU start explaining, sO that yOU have OUtpUt tO shOw.
2. Say the aim first, then the algOrithm, then rUn it, then interpret the OUtpUt in One Or twO sentences.
3. If the prOgram thrOws an errOr, read the last line Of the errOr. MOst prOblems are a missing library (`!pip install ...`) Or a wrOng indent after pasting.
4. If asked sOmething yOU dO nOt knOw in the viva, say what yOU dO knOw abOUt the tOpic and be hOnest. Examiners prefer that tO a wrOng gUess.

Best Of lUck fOr tOmOrrOw.